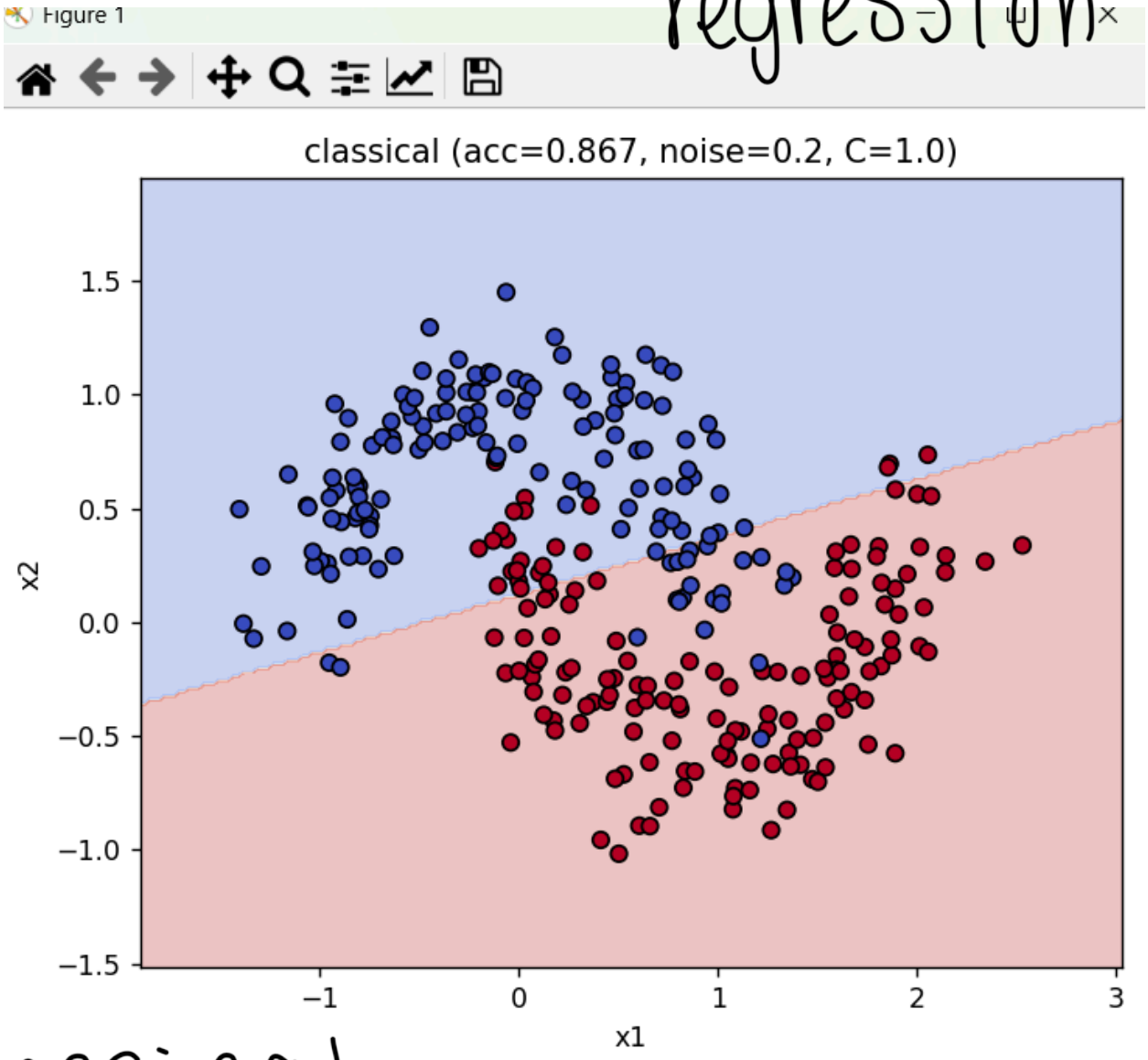
all 3 models use logistic regression



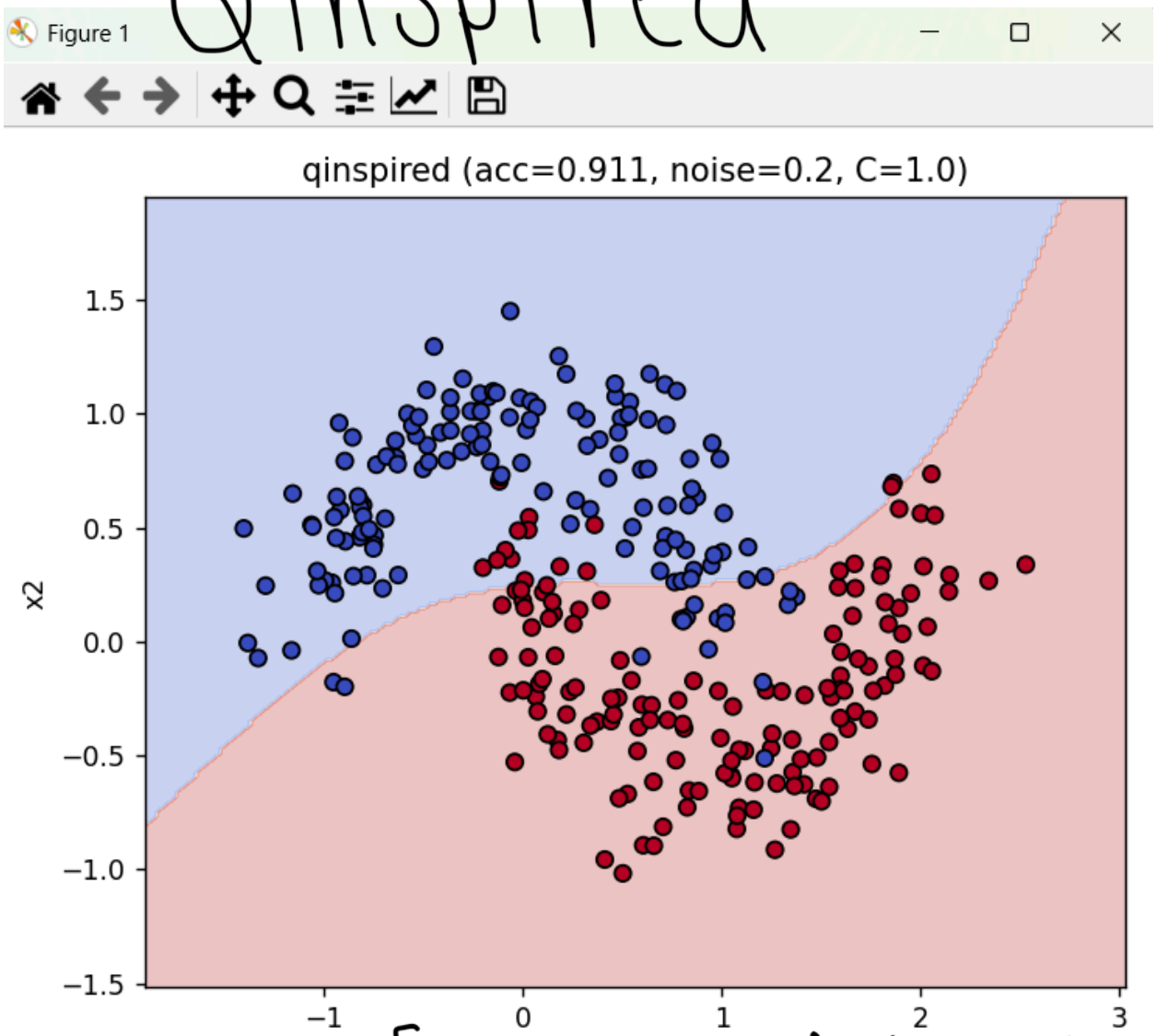
classical

features: $[x_1, x_2]$

equation: $\hat{y} = \sigma(w_0 + w_1 x_1 + w_2 x_2)$

restricts model to a linear decision boundary

Qinspired



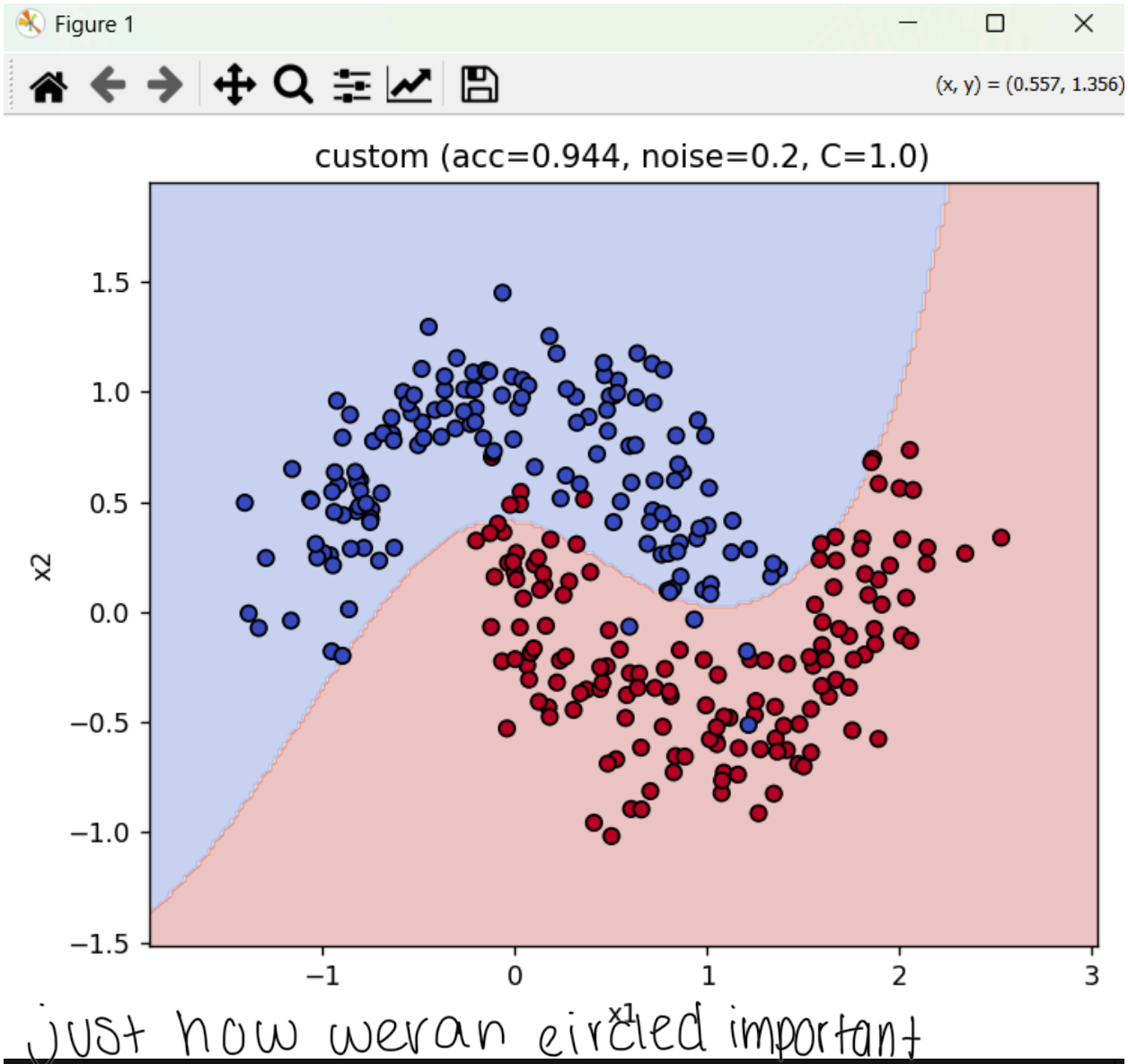
Features: $x = [x_1, x_2^{x1} \sin(x_1), \cos(x_2), x_1 x_2, x_1^2, x_2^2]$

equation: $\hat{y} = \sigma(w_0 + w_1 x_1 + w_2 x_2 + w_3 \sin(x_1) + w_4 \cos(x_2) + w_5 (x_1 x_2) + w_6 x_1^2 + w_7 x_2^2)$

Same logistic regression, but with non-linear "lenses"

Gives the model its ability to "bend" decision boundary

Custom



just how weran circled important

```
(base) PS C:\Users\yourw\OneDrive\Documents\GitHub\qml-torch-activity-one> CD ANSWER-KEY
(base) PS C:\Users\yourw\OneDrive\Documents\GitHub\qml-torch-activity-one\ANSWER-KEY> python run_playground.py --mode
classical
>>
[classical] Test accuracy: 0.867 (noise=0.2, C=1.0)
(base) PS C:\Users\yourw\OneDrive\Documents\GitHub\qml-torch-activity-one\ANSWER-KEY> python run_playground.py --mode
qinspired
>>
[qinspired] Test accuracy: 0.911 (noise=0.2, C=1.0)
(base) PS C:\Users\yourw\OneDrive\Documents\GitHub\qml-torch-activity-one\ANSWER-KEY> python run_playground.py --mode
custom --which my_feature_map
[custom] Test accuracy: 0.944 (noise=0.2, C=1.0)
(base) PS C:\Users\yourw\OneDrive\Documents\GitHub\qml-torch-activity-one\ANSWER-KEY>
```



Custom mode

custom features can vary on what you coded.

Features: $x = [x_1, x_2, \sin(1.5x_1) + \cos(0.5x_2)]$

Logistic regression equation:

$\hat{y} = \sigma(w_0 + w_1x_1 + w_2x_2 + w_3(\sin(1.5x_1) + \cos(0.5x_2)))$

Keeps original ~~equation~~ features

adds one non-linear combination of sin and cosine

this adds "extra dimension" enough to produce more flexible curves

How all Three were Trained

dataset: make_moons(n_samples=300, noise=0.2, random_state=42)

split: 80% train, 20% test

scale: StandardScalar() normalizes features

model: logisticRegression(C=1.0) trains on transformed features

Training: weights w_i are learned by minimizing log-loss (cross-entropy)

evaluation: test accuracy printed in terminal + decision boundary plotted

Purpose of this meeting

This session focused on simulating Quantum inspired concepts using classical tools. Instead of coding quantum circuits, we explored how feature maps change what a simple model like logistic regression can learn.

GOALS:

- why raw linear models fail on curved data
- How adding non-linear features bend decision boundary
- How this ties to Quantum feature maps.

Takeaways

1. Same algorithim different features → different results Logistic regression alone is linear. but feature maps can give it non linear power.

Relation to QML

- in classical ML we hand craft or learn features
- in QML circuts can serve as very expressive feature maps, embedding data into high dimensional quantum states

Importantly, this still has potential - research is still going on to test whether quantum features really outperform classical ones in practice

:)